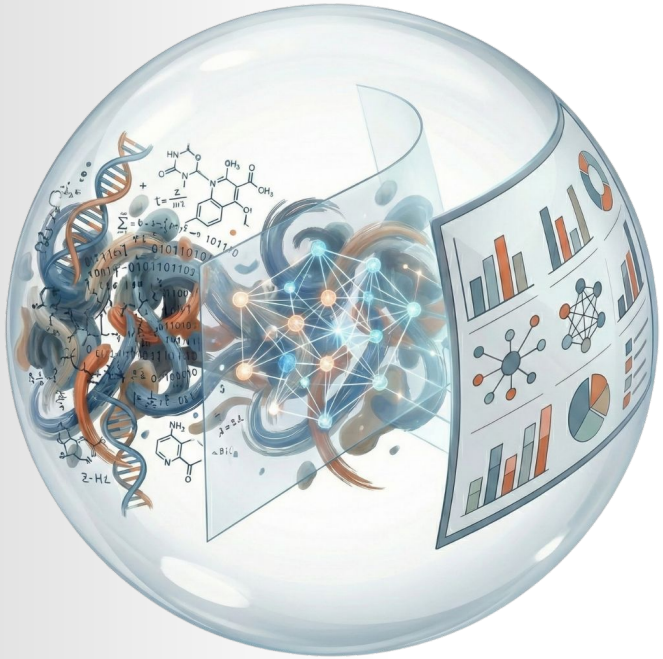


Introduction to Web Developement and *Promptotyping* with Claude Code

Project: Legal History Hub
Max-Planck-Institut für Rechtsgeschichte & Rechtstheorie
Workshop 2 - 13.03.2026

Christian Steiner
Digital Humanities Craft OG
www.dhcraft.org | office@dhcraft.org

Workshop Overview

1. Quick Start into Web Development
2. Git
3. AI / Vibe / Agentic … Coding
4. Promptotyping
5. Live Demo
6. Claude Code 101

Coding with AI - a reasonable Setup

- VS Code with Live Server Extension
- Coding Agent:
  - https://claude.com/product/claude-code
- Install Python and https://nodejs.org/
- Basic handling of Browser as Web Development Tool
- GitHub Account and GitHub CLI
- Knowledge Organization with .md: https://obsidian.md/ (optional)
- File transformation and preparation: https://www.docling.ai/ (optional, “PDF → Markdown”!)

Quickstart: How does the Web work?

Your browser (the client) sends a request to a server. The server sends a response back. The protocol for this is called HTTP.

What comes back are text files: HTML for structure, CSS for appearance, JavaScript for interactivity. Your browser assembles these and displays them as a web page.

URL: The address of a resource on the web. `https://example.com/page.html`

HTTP: Hyper Text Transfer Protocol. The protocol browsers and servers use to communicate. Request → Response.

Client: Your browser. It sends requests and displays the results.

Server: A computer somewhere on the internet that serves files. It waits for requests and responds.

HTML, CSS, JS: The three languages your browser understands. Exactly the ones you're learning this semester, and exactly the ones your coding agents will write for you.

Internet vs. Web

1. **Internet:** the infrastructure. Cables, routers, computers, connected worldwide.
2. **Web:** a service that runs on top of the internet. Web pages, links, browsers. What you're building.
3. The internet has existed since the 1960s. The web since 1991.

URL

The address of a resource on the web.

`https://example.com:443/path/page.html?search=cat#result`

1. `https` – the protocol. The *s* stands for "secure": the connection is encrypted. Without it (`http`), everything is transmitted in plain text. Today, `https` is the standard.
2. `example.com` – the domain (resolved to an IP address via DNS)
3. `:443` – the port (usually invisible because it's the default)
4. `/path/page.html` – the path to the file
5. `?search=cat` – a parameter
6. `#result` – an anchor, a jump target on the page

Frontend und Backend

1. **Frontend:** What happens in the browser. HTML, CSS, JavaScript. This is what your users see.
2. **Backend:** What happens on the server. Databases, logic, authentication. Invisible to the user.
3. **Our project:** Frontend. But Claude Code can also write backend code if another project requires it.

Is HTML a programming language?

1. **No**. HTML describes structure; it doesn't compute anything. It is a markup language, just like Markdown or XML.
2. **Programming languages**: JavaScript, Python, Java … Markup languages: HTML, XML, Markdown …
3. **The distinction**: HTML tells the browser *what to display*. JavaScript tells it *what to do*.

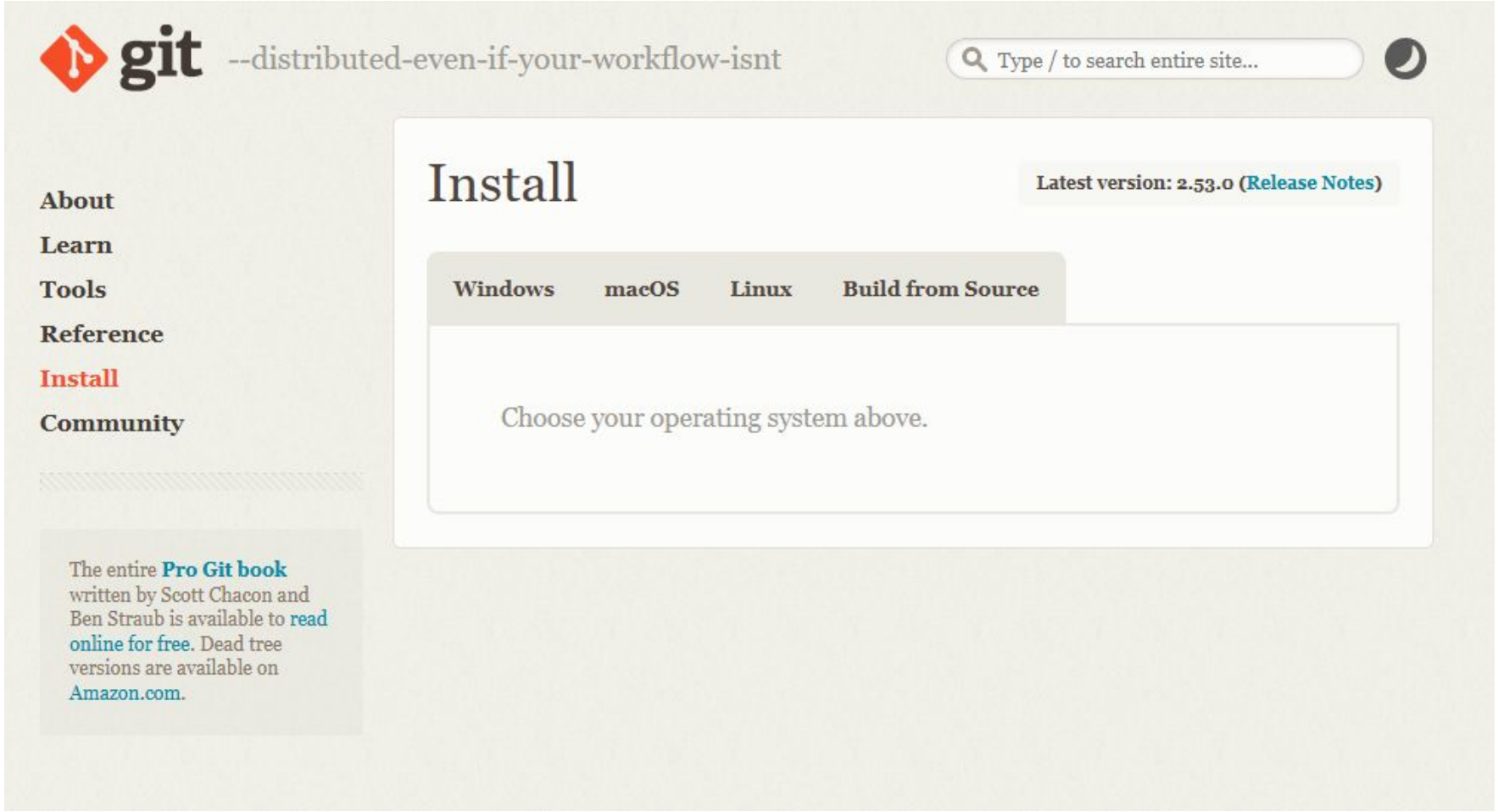
What is Git and why do you need it?

- Version control for code. Every change is saved; nothing is lost.
- You can go back to any previous state at any time. You can see what changed and when. Multiple people can work on the same project without overwriting each other's work.
- Git runs locally on your machine. GitHub/GitLab is the platform where your project lives online ("remote").
- Your coding agents (Claude Code, Copilot, Codex) all work with Git. Commits, branches, pull requests: agents often handle these automatically for you, but you need to understand what's happening.

Git: Key Concepts

- **Repository (Repo):** Your project folder, tracked by Git. Contains the entire history.
- **Clone:** Copying a repo from GitHub ("remote") to your local machine.
- **Staging:** Marking changes to be included in the next commit. Like a shopping cart before checkout.
- **Commit:** A saved snapshot. Includes a short description of what changed.
- **Push:** Uploading your commits from your machine to GitHub ("remote").
- **Pull:** Fetching changes from GitHub to your local machine.
- **Branch:** A parallel line of work. You work on a feature without affecting the main codebase.
- **Merge:** Combining a branch back into the main branch.
- **Pull Request (PR):** A request on GitHub to merge your branch. Others can review and comment on the code before it's merged. ("Merge Request" on GitLab)

https://git-scm.com/install/



Coding Agents can install things for you!

What does "AI / Vibe / Agentic … Coding" mean?

Vibe Coding means trying out LLM-generated code without understanding it in detail.

Informed Vibe Coding (?) adds the ability to evaluate results, spot errors, and make targeted improvements.

“Traditional”

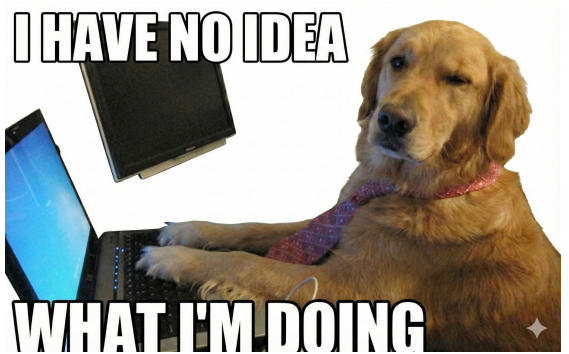
- Writing code, learning syntax, …
- Investing a lot of time, failing a lot – BUT truly understanding it in the end

LLM-assisted (Agentic Coding)

- Formulating requirements, evaluating results, …
- Very exciting results very quickly – with the lingering feeling: what am I actually doing here?

2025 LLM Year in Review.
https://karpathy.bearblog.dev/year-in-review-2025/

Simon Willison’s Weblog-
https://simonwillison.net/2025/Mar/19/vibe-coding



AI Coding Literacy? – A Proposal

Computational Thinking: Understanding how to solve problems with code

Requirement Engineering: Precisely formulating what you need

Context Engineering: Giving the LLM the right information in its context window

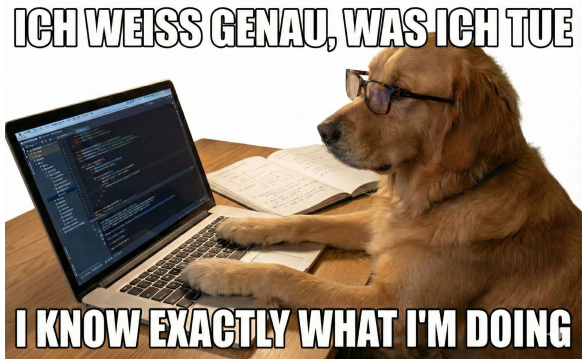
Prompt Engineering: Operationalizing prompting strategies for your own problems

Code Literacy: Being able to read and evaluate generated code

Review: Systematically verifying results

Domain Expertise

…



Computational Thinking

"*Computational thinking involves **solving problems, designing systems**, and understanding **human behavior**, by drawing on the concepts fundamental to computer science. [...] Thinking like a computer scientist means **more than being able to program a computer**.*"

For working with LLMs, this means: breaking problems down.

Without Computational Thinking: "Build me a website for my data"

With Computational Thinking:

1. Understand the data – What columns are there?
2. Export the data – Excel → CSV
3. Load the data – CSV
4. Display the data – Generate a table
5. Add search – A text field that filters

Wing, J. M. (2006). Computational Thinking. Communications of the ACM, 49(3), 33-35. https://dl.acm.org/doi/10.1145/1118178.1118215

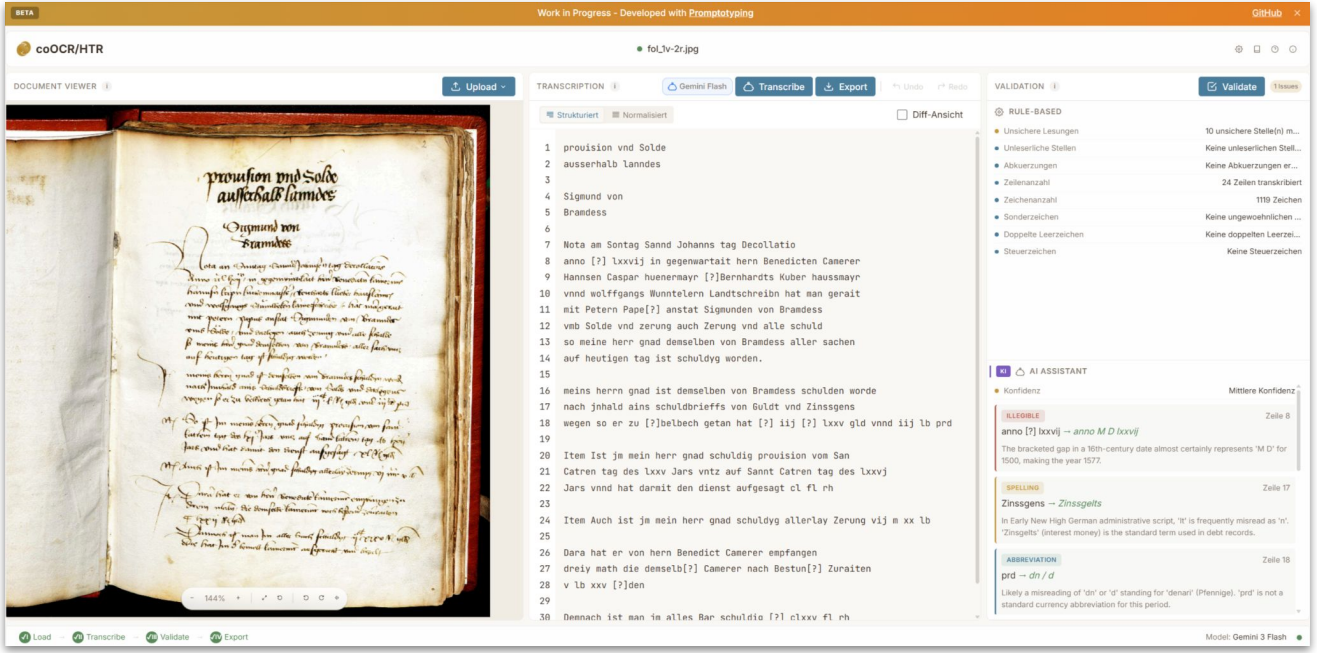
“*Roll up your sleeves to not fall behind*”

> “*There's a **new programmable layer of abstraction** to master involving **agents, sub-agents, prompts, contexts, memory, tools, plugins, skills, hooks** ... and a need to build an all-encompassing **mental model for strengths and pitfalls of fundamentally stochastic, fallible, unintelligible entities**.*”

— Andrej Karpathy, Dezember 2025

What's going on with Claude Code?. https://www.youtube.com/watch?v=pwmK46hhfwg

coOCR/HTR: Built with *Promptotyping* & Agentic Coding



Editor-in-the-Loop tool for OCR/HTR workflows

Browser-based, open source, open development.

Research Preview (Beta) — built and actively developed using the ***Promptotyping*** as **Context Engineering** method

Built with **Claude Code + Opus 4.5 & 4.6** Promptotyping

Demo: https://dhcraft.org/co-ocr-htr
GitHub Repo: https://github.com/DigitalHumanitiesCraft/co-ocr-htr
Promptotyping documents: https://github.com/DigitalHumanitiesCraft/co-ocr-htr/tree/main/knowledge

Asymmetric Amplification

Why AI Does Not Automate Research —
But Disruptively Amplifies Computer-Based Research Work.

https://dhcraft.org/excellence/blog/Asymmetric-Amplification

Automation Narrative

"If I look at coding, programming, which is one area where AI is making the most progress, what we are finding is we are not far from a world – I think we'll be there in three to six months – where AI is writing 90% of the code."

And then in 12 months, we may be in a world where AI is writing essentially all of the code.

— Dario Amodei. "The Future of U.S. AI Leadership with CEO of Anthropic Dario Amodei." https://www.youtube.com/live/esCSpbDPJik

Dario Amodei — "We are near the end of the exponential". https://youtu.be/N5JDzS9MQYI

The Two Best AI Models/Enemies Just Got Released Simultaneously.
https://youtu.be/1PxEziv5XIU?si=vmVMA4n_48Cp47sR

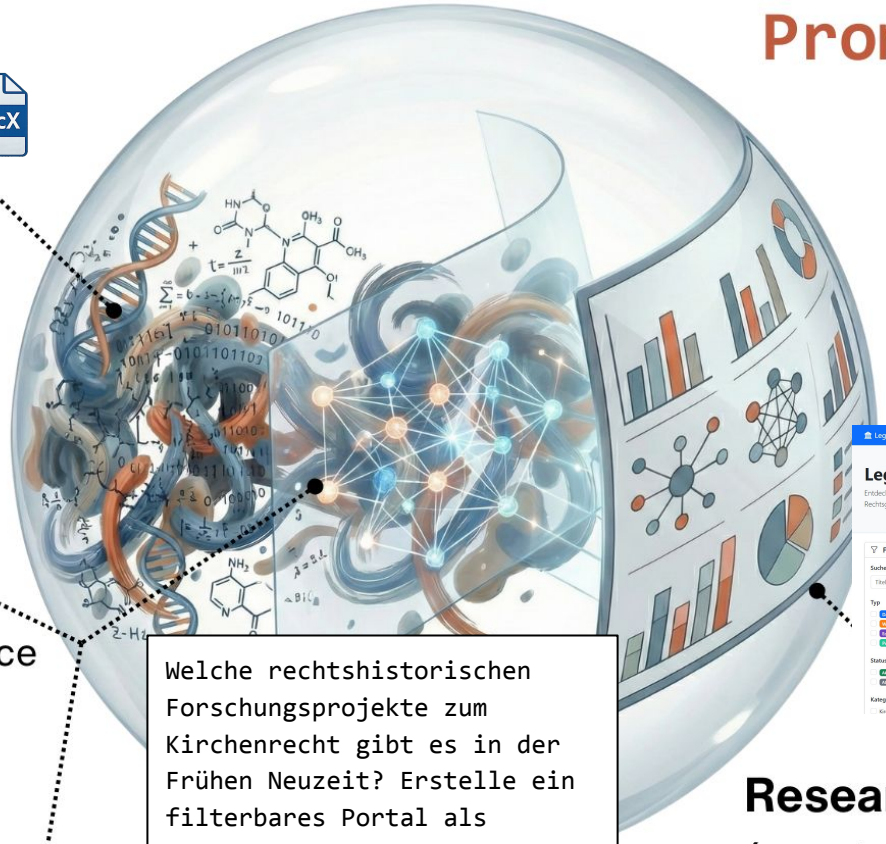
Claude AI Co-founder Publishes 4 Big Claims about Near Future: Breakdown. .
https://youtu.be/Iar4yweKGoI?si=IBsG9IZWWtzVtOWb

Research Data

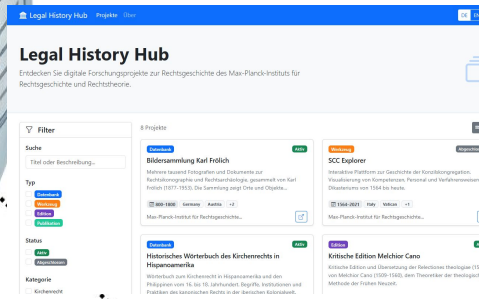
Promptotyping

Research Domain & Expert in the Loop

Co-Intelligence (Mollick)



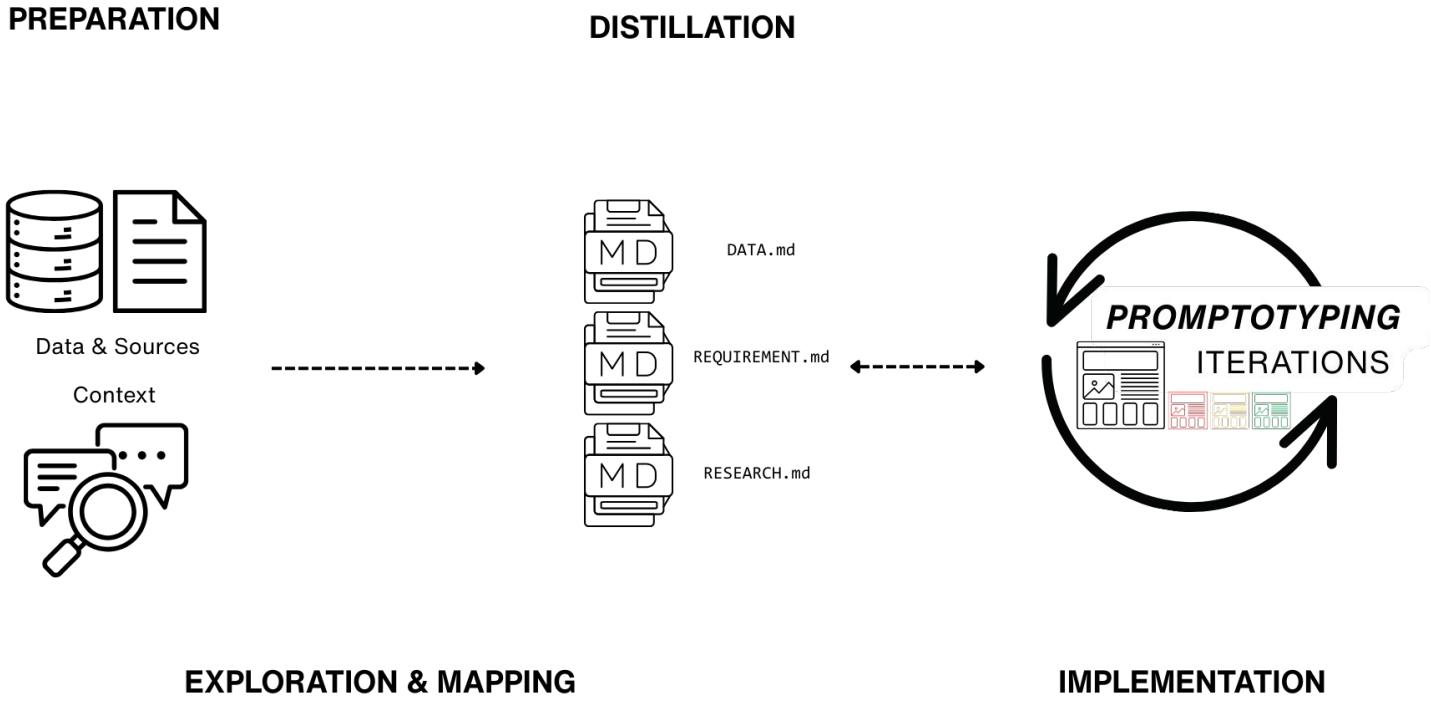
```
Welche rechtshistorischen Forschungsprojekte zum Kirchenrecht gibt es in der Frühen Neuzeit? Erstelle ein filterbares Portal als statische Website auf Basis dieser CSV-Metadaten.
```



(Frontier-)LLM & Context Engineering

Research Artefacts

(e.g. tools, workflows, models)



Promptotyping - extremely fast, researcher-centred, and research-data-driven creation of prototypes for research tools, workflows, and data-models using frontier LLMs and Context Engineering techniques.

Context Window, Context Rot and Destillation

Context Window – The maximum number of tokens an LLM can process in a single pass. Functions like a working memory that exists only for the duration of the interaction. Claude Opus 4.5 processes up to 200,000 tokens, but the limit is not the goal.

Context Rot – LLM performance degrades as context length increases. Unstructured text acts as noise, diverting attention from relevant information. More context does not automatically mean better results.

Context Engineering – The systematic design of what goes into the context window. Selection, compression, and arrangement of information with the goal of maximizing model response quality under limited resources.

Distillation (Context Compression) – The practical application of context engineering. Knowledge is condensed into compact Markdown documents. Maximum information with minimal tokens.

Vaswani, Ashish, et al. "Attention Is All You Need". Advances in Neural Information Processing Systems, Bd. 30, 2017. https://arxiv.org/abs/1706.03762

Hong, Kelly, Anton Troynikov, und Jeff Huber. Context Rot: How Increasing Input Tokens Impacts LLM Performance. Chroma, 2025. https://research.trychroma.com/context-rot

Mei, Lingrui, Jiayu Yao, Yuyao Ge, et al. "A Survey of Context Engineering for Large Language Models". arXiv:2507.13334. Preprint, arXiv, 21. Juli 2025. https://doi.org/10.48550/arXiv.2507.13334

Anthropic. "Effective context engineering for AI agents". Anthropic Engineering Blog, September 2025. https://www.anthropic.com/engineering/effective-context-engineering-for-ai-agents

Critical Expert in the Loop

LLMs have no internal mechanisms for verification. Responsibility for correctness lies with the person working with the model. Critical evaluation requires domain knowledge. Only those who know the subject matter can judge whether the model responds plausibly or confabulates.

Structural weaknesses

- **Sycophancy:** LLMs tend to agree with users rather than contradict them.
- **Confabulation (hallucinations):** LLMs generate plausible-sounding but fabricated details.
- **Non-determinism:** The same prompt yields different results. Variance is inherent to the system, not a bug.

Questions to ask during work

- Did the model challenge my assumption, or merely confirm it?
- Can I independently verify specific claims such as numbers, names, or sources?
- Did I ask in a leading way, or did I leave room for disagreement?
- What happens if I submit the same prompt again?

Metacognition – The Critical Expert in the Loop reflects on both output and process.

Pollin, Christopher. 'Promptotyping mit Claude Sonnet 4. Vibe-Coding erfordert den Critical-Expert-in-the-Loop'. Digital Humanities Craft - Research Blogs, 27 May 2025. https://dhcraft.org/excellence/blog/Critical-Vibing-Claude-4/.

Malmqvist, Lars. „Sycophancy in Large Language Models: Causes and Mitigations". Preprint, 22. November 2024. https://arxiv.org/abs/2411.15287v1 .

Ji, Ziwei, et al. „Survey of Hallucination in Natural Language Generation". ACM Computing Surveys, Bd. 55, Nr. 12, 2023. https://arxiv.org/abs/2202.03629.

Context Engineering: Distilling Knowledge into Markdown Documents

Instead of explaining everything to the LLM from scratch in every prompt, we distill our knowledge about a dataset into a structured Markdown document. This document is provided to the LLM as context.

Why?

- Reusable across multiple chats (prevents Context Rot)
- Compact and token-efficient
- Forces us to understand our own data (Expert-in-the-Loop)
- Easy to do and effective!

Context Files for Agentic Systems

AGENTS.md - An open Markdown format for steering coding agents, maintained under the Linux Foundation. Contains project-specific context: build commands, test requirements, style guidelines. Tool-agnostic, supported by Claude Code, Cursor, Copilot, Gemini CLI, Windsurf, and others.

CLAUDE.md - Configuration file for Claude Code in the repository root. Defines code style, review criteria, and project rules. Loaded into the system prompt at session start; hierarchical from enterprise to subdirectory level.

GEMINI.md - Google's equivalent for Gemini Code Assist. Same principle, vendor-specific filename.

All formats follow the same pattern: distill project knowledge into Markdown that structures the agent's context. AGENTS.md is the emerging cross-tool standard; the others are vendor-specific. In practice, teams symlink between them.

Promptotyping-Documents

```
knowledge/
├── data.md
├── design.md
├── implementation.md
├── journal.md
├── knowledge.md
└── requirements.md
```

Promptotyping Documents are Markdown files that distill knowledge about research data, domain, and requirements. They are created during the Exploration and Distillation phases and serve as compact, curated context for working with LLMs.

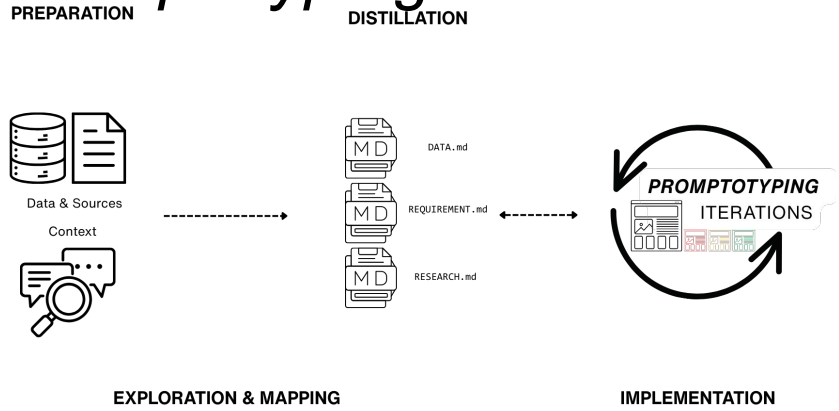
1. Preparation: Gather all relevant materials: data, documentation about the data, standards, models, research questions, domain knowledge, secondary literature, etc.

2. Exploration & Mapping: Use the LLM to explore what can be done with these data and this context. Make connections between research questions and data structures explicit. The result is knowledge, not documents.

3. Distillation: Condense maximum information with minimal tokens into Promptotyping Documents. The knowledge from Preparation and Exploration is compressed into curated Markdown files.

4. Implementation: Generate a prototype using the Promptotyping Documents as context. The LLM generates, the Critical Expert validates. New knowledge feeds back into the documents.

The 4 Phases of *Promptotyping*



Element	Markdown Syntax

Heading	`# H1`
	`## H2`
	`### H3`

Bold	`**bold text**`
Italic	`*italicized text*`
Blockquote	`> blockquote`

Ordered List	`1. First item`
	`2. Second item`
	`3. Third item`

Unordered List	`- First item`
	`- Second item`
	`- Third item`

Code	`` `code` ``
Horizontal Rule	`---`
Link	`[title](https://www.example.com)`
Image	`![alt text](image.jpg)`

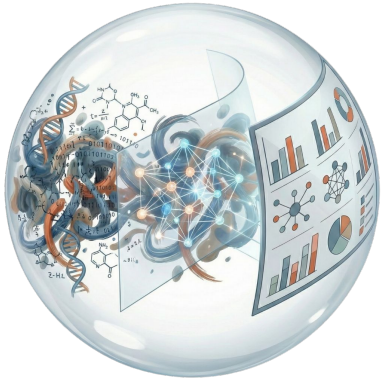
https://www.markdownguide.org

Markdown

Lightweight markup language for text formatting, ensuring maximum readability in raw format and easy conversion to structured formats (HTML, PDF, LaTeX …).

Good for LLMs because:

- High presence in training data
- Token-efficient
- Separates structure from content (e.g., expressing subsections or hierarchies using lists)
- Natively used by Companies (see Agent.md and Skills.md …)



Live Demo

Promptotyping

Extract information from source and “compress” source structure into a DATA.md

Analyze this excerpt from the Laban Morey Wheaton Day Book (1828-1832), a historical accounting ledger from Norton, Massachusetts.

```
[1]

Monday September-15th-1828
Settled                 Derius Drake DR
To ax dlvd By Puffer 9/1                1                       50
Settled                 By cutting two sticks Cr
109                     Pliny Puffer Cr
By 1 ax dlv.d. Drake 9/                 1                       50
342                     Asa Danforth Cr
By drawing two logs from Rogers
Monday Sept. 22d 1828
342                     Asa Danforth Cr
By 1 Days work with oxen, wheels, &
Settled                 Derius Drake D
To order upon T. Smith for 3/                                   50
"
By 1 Days work Cr
October 3d 1828
25                      Thomas Danforth 2d
To 1 lb. 4d Cut Nails .8
" 1 Buss. Rye 90t 90                    98
Settled                 Nathaniel Lincoln D
To cash 8/                              50
342                     Asa Danforth C
By drawing two sticks from Mss Bowend
4.
139                     John Deane D
To cash 6/              1               00

[...]
```

Extract and structure all entities (like persons, goods/services, places, dates, {what else do exist}), identify all transactions (type, direction, amount, status, {what else do exist}), map the relationships, and note any ambiguities requiring historical interpretation. Use a compact and formal style.

Follow-Up Prompt

How can we accurately, concisely and formally represent the structure of the source in a Markdown document? It needs to represent all information.

Describe the data in a more general way. How can the structure of the source be accurately, concisely and formally represented in a Markdown document? It must represent all the information.

Ist it all correct and perfect? List and explain!

Create the DATA.md.

You have to adjust the follow-up prompts and also control the target. It can be useful to have examples or to describe it in more general terms.

Hands-On: Acquire (a lot of) Data, Explore, Generate data.md and requirements.md

Dataset: MoMA Collection Data (Museum of Modern Art, New York). Over 160,000 artworks. CC0 license.
github.com/MuseumofModernArt/collection

Step 1. Set up environment. Create a project folder, open it in VS Code. Claude Code is already available in the sidebar.
Step 2. Acquire data. Ask Claude Code: `download Artworks.csv from github.com/MuseumofModernArt/collection into this folder`.
Step 3. Explore data. Ask Claude Code to explore the dataset for you: `analyze Artworks.csv, show me the columns, data types, value ranges, and a few sample rows`. Claude Code will write and run the script itself. No copy-pasting needed.
Step 4. Iterate. Ask follow-up questions directly: `how many artworks have no date? which nationalities appear most often? show me the distribution of mediums`. Claude Code runs each analysis and returns the results directly in the chat.
Step 5. Generate data.md. Ask Claude Code: `based on your exploration, generate a data.md that documents this dataset – structure, columns, value ranges, data quality issues, and notable patterns`.
Step 6. Review the file, ask for corrections: `the Date column also contains date ranges like "1920-1925", add that to data.md`.
Step 67 Explore the possibility space. Ask Claude Code: `read data.md and propose user stories for a web-based research interface, published as a static site with a single index.html using vanilla JavaScript and CSS. Write the result as requirements.md`. Review and refine.

By the end of this hands-on you have two knowledge documents: a `data.md` that describes your dataset and a `requirements.md` that maps what could be built with it. We will use these to create a web tool later.

DATA.md – MoMA Collection Artworks

1. Identität und Herkunft

Quelle. Museum of Modern Art (MoMA), New York. Öffentlich zugänglicher Sammlungsdatensatz.
Datei. `Artworks.csv`
Umfang. Über 160.000 Einträge (exakte Zahl durch Profiling zu bestätigen).
Granularität. Eine Zeile pro Sammlungsobjekt. Werkgruppen und ihre Einzelteile können als separate Zeilen koexistieren (siehe AccessionNumber).
Lizenz. Zu prüfen; MoMA veröffentlicht Sammlungsdaten üblicherweise unter CC0 auf GitHub.

---

2. Feldschema

Jedes Feld ist einmal dokumentiert, einschließlich Typ, Kodierung und struktureller Besonderheiten. "Typ (CSV)" beschreibt, was Pandas beim Einlesen erkennt, nicht die Semantik.

Gruppe	Feld	Typ (CSV)	Kodierung und Besonderheiten
	Werk	Title	object	Freitext. Kann Ortsangaben, Projektnamen und Untertitel enthalten. Teilweise sehr lang. Inkonsistente Leerzeichen beobachtet (`"New York, New York , Episode"`, ObjectID 21).
	ObjectID	int64	Fortlaufend. Dient als Primärschlüssel.
	AccessionNumber	object	Hierarchisches Schema, z.B. `3.1995.1-24` (Gruppe) vs. `3.1995.1` (Einzelblatt). Die Gruppenpräfixe bezeichnen vermutlich einen Erwerbungsvorgang, die genaue Nummerierungslogik ist aber nicht belegt. **Konsequenz.** Ein physisches Werkensemble kommt sowohl als Gruppe als auch in Einzeleinträgen vor. Ob Gruppen- oder Einzeleinträge gezählt werden, muss pro Analyse entschieden werden.
	URL	object	Aufbau `https://www.moma.org/collection/works/{ObjectID}`. Vollständig befüllt im Sample.
	ImageURL	object	Link zum Digitalisat. Nicht für alle Werke vorhanden.
	Person	Artist	object	Name. Denormalisiert, pro Zeile wiederholt. Für personenbezogene Analysen empfiehlt sich Normalisierung über ConstituentID. Ob bei Kollaborationen mehrere Namen oder Zeilen vorliegen, ist offen.
	ConstituentID	int64	Eindeutige Personen-ID. Ermöglicht Verknüpfung über Werke hinweg.
	ArtistBio	object	Zusammengesetzter String, z.B. `(American, born Estonia. 1901-1974)`. Enthält Geburtsland und Migrationshintergrund, die in den Einzelfeldern verloren gehen. Für Geburtsland-Analysen ergiebiger als Nationality.
	Nationality	object	**Klammernkodierung.** z.B. `(Austrian)`. Leere Klammern `()` statt NULL bei fehlender Angabe. Klammern müssen beim Parsing entfernt werden. Ob das Vokabular kontrolliert ist, durch Profiling zu klären.
	BeginDate	object	**Klammernkodierung.** Geburtsjahr als String, z.B. `(1841)`. `pd.to_numeric` scheitert an den Klammern; Stripping nötig vor Konvertierung.
	EndDate	object	**Klammernkodierung + Sentinel.** Todesjahr, z.B. `(1918)`. Wert `(0)` für lebende Künstler*innen statt NULL. Muss als fehlend behandelt werden.
	Gender	object	**Klammernkodierung.** z.B. `(male)`. Im Sample ausschließlich `(male)`. Wertespektrum durch Profiling zu klären.
Beschreibung	Date	object	**Freitext mit implizitter Struktur.** Beobachtet: Einzeljahr (`1890`), Kurzspanne (`1976-77`). Erwartet: Langspanne, Circa, Dekade, Komplex (`designed 1925, executed 1926`). Für zeitliche Analysen ist ein Parser erforderlich.
	Medium	object	**Freitext mit impliziter Struktur.** Kommagetrennte Materialien, Träger nach `on`. z.B. `Graphite, pen, color pencil, ink, and gouache on tracing paper`. Nicht normalisiert.
	Dimensions	object	**Freitext, Mischformat.** Zoll und cm kombiniert, cm-Werte redundant zu Height/Width. Bei Werkgruppen Präfixe wie `Each:`.
	Classification	object	z.B. `Architecture`. Im Sample konsistent, Vokabularkontrolle über Gesamtdatensatz durch Profiling zu prüfen.
Verwaltung	Department	object	z.B. `Architecture & Design`. Selbe Einschränkung wie Classification.
	CreditLine	object	Freitext. Schenkungen, Ankäufe, Mischformen.
	DateAcquired	object	**Einziges standardisiertes Datumsfeld.** Format `YYYY-MM-DD`. Sauber nach datetime konvertierbar.
	Cataloged	object	Flag. Im Sample ausschließlich `Y`. Wert `N` plausibel, aber nicht beobachtet.
	Onview	object	Im Sample durchgehend leer. Möglicherweise nicht gepflegt.
Maße	Height (cm)	float64	Befüllt für 2D-Werke im Sample. Ob insgesamt bestbesetzt, ist plausibel, aber Sample-Bias (nur Architecture).
	Width (cm)	float64	Siehe Height.
	Depth (cm)	float64	Dreidimensionale Objekte. Im Sample leer.
	Circumference (cm)	float64	Im Sample leer.
	Diameter (cm)	float64	Im Sample leer.
	Length (cm)	float64	Im Sample leer.
	Weight (kg)	float64	Im Sample leer.
	Seat Height (cm)	float64	Möbelobjekte. Im Sample leer.
	Duration (sec.)	float64	Zeitbasierte Medien. Im Sample leer.

Erwartung zu Maßfeldern. Füllrate stark Classification-abhängig. Mehrzahl der Spalten vermutlich dünn besetzt.

Fehlende Relationen. Keine Felder für Ausstellungshistoria, Provenienz (über CreditLine hinaus), Werkbeziehungen oder kuratorische Einordnung.

---

3. Datenqualitätsprobleme

Problem	Feld(er)	Beispiel	Status
Sentinel statt NULL	EndDate	`(0)` für Lebende	bestätigt
Leere Klammern statt NULL	Nationality	`()`	bestätigt
Klammern verhindern numerisches Parsing	BeginDate, EndDate, Nationality, Gender	`(1841)` → NaN bei `pd.to_numeric`	bestätigt
Inkonsistente Leerzeichen	Title	`"New York, New York , Episode"`	bestätigt
Werkgruppen-Einzelwerk-Doppelung	AcessionNumber	`3.1995.1-24` neben `3.1995.1` bis `.24`	bestätigt
Redundante Maßangaben	Dimensions vs. Height/Width	cm-Werte doppelt	bestätigt
Mehrere Künstler*innen pro Werk	Artist, ConstituentID	Kollaborationen	erwartet
Fehlende Digitalisate	ImageURL	Lücken	erwartet
Ungleiche Maß-Füllrate nach Werkklasse	alle Maßspalten	Duration nur Medienkunst	erwartet
Uneinheitliche Gender-Kodierung	Gender	nonbinary, Gruppen, fehlend	erwartet
Schreibvarianten in kategorialen Feldern	Classification, Department	Umbenennungen	erwartet
Heterogene Data-Granularität	Date	Einzeljahr vs. Dekade je nach Department	erwartet

---

4. Analytisches Potenzial und offene Fragen

Sammlungspolitik über Zeit. DateAcquired × Department × Classification. *Offen:* Verteilung von DateAcquired über die Jahrzehnte?*

Demografie der Künstler*innen. Gender, Nationality, Lebensdaten. *Offen:* Wie viele eindeutige ConstituentIDs? Welche Gender-Werte jenseits von `(male)`?*

Abteilungen und Klassifikationen. Kreuztabelle Department × Classification. *Offen:* Vollständige Wertelisten und Verteilungen?*

Medien und Materialien. Längsschnitt nach Parsing von Date und Medium. *Einschränkung:* NLP oder regelbasiertes Parsing nötig.*

Zeitliche Verteilung der Werkdatierung. Erfordert Date-Parser. *Einschränkung:* Circa-Angaben und Spannen erzeugen Unschärfe.*

Geburtsland vs. Nationalität. ArtistBio ist reicher als Nationality. *Einschränkung:* String-Parsing nötig.*

Physische Charakteristiken. Maßfelder nach Classification. *Offen:* Füllraten der Maßspalten pro Werkklasse?*

Werkgruppen-Struktur. AccessionNumber-Parsing. *Offen:* Wie viele Gruppeneinträge, und wie verzerren sie Zählungen?*

Digitalisierungsgrad. ImageURL-Füllrate. *Offen:* Anteil ohne ImageURL? Korrelation mit Department oder Alter?*

Nicht machbar ohne zusätzliche Daten. Ausstellungsfrequenz, Marktwert, Provenienzforschung, kuratorische Kontextualisierung.

data.md

Some Follow-Up Prompts:

- Is everything correct? List and explain.
- Can the same information be presented more compactly without losing detail or complexity? List and explain.
- Write the improved and compact knowledge document data.md

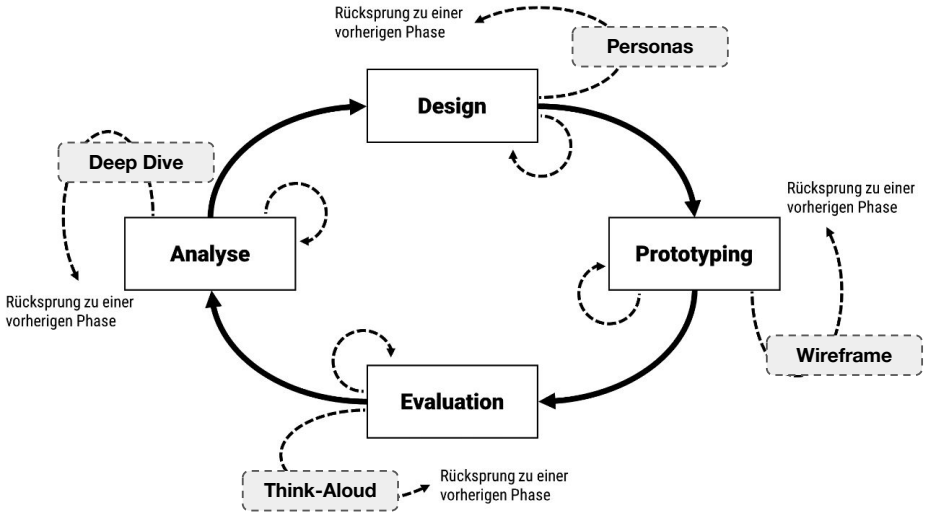
```
# requirements.md — MoMA Collection Research Interface

## 1. Product Vision

A static, browser-side research tool for exploratory analysis of the MoMA collection dataset (~160,000 entries). Single `index.html` file, vanilla JavaScript and CSS, no frameworks. The CSV is loaded and parsed client-side.

The interface serves researchers who want to detect patterns, generate hypotheses, and export subsets. It complements rather than replaces the official MoMA collection browser by adding analytical perspectives.


## 2. Prerequisites

These are not features but preconditions. Without them, no analytical question can be answered reliably.

**FP1 — Field cleanup on load.** Strip parentheses from Nationality, BeginDate, EndDate, and Gender. Treat EndDate value `0` as missing. Convert BeginDate and EndDate to numeric. Parse DateAcquired as Date object.
**FP2 — Work group detection.** Parse AccessionNumber to distinguish group entries (e.g. `3.1995.1-24`) from individual entries (e.g. `3.1995.1`). Set a boolean flag `isGroupEntry` per row. Default counts include group entries and exclude their constituent parts, since the group entry represents the logical work unit. A toggle allows switching to individual-entry counting. *Rationale — The group entry is the summary object; counting individual parts instead would inflate totals.*
**FP3 — Date field parsing.** Convert the free-text Date field to a numeric approximation (earliest year). Recognized patterns include single year (`1896`), short span (`1876–77`), long span (`1876–1880`), circa (`c. 1900`). Mark unparseable values as missing and track their proportion. *Limitation — complex entries like "designed 1925, executed 1926" reduce to the first recognized year. All time-based visualizations must display the share of unparseable values.*
**FP4 — Performance strategy.** Pre-compute and cache aggregations (counts, distributions) on initial load. Limit table rendering to 500 rows with pagination. Show a loading indicator during initial parse. *Note — whether filtering is incremental or full-recompute is an implementation decision, not a requirement.*
**FP5 — Multi-artist handling.** ConstituentID may contain comma-separated values for collaborative works. The loader must split these and associate each work with all its constituent IDs. Artist counts must account for this to avoid undercounting. *Status — not observed in the sample but documented as expected in DATA.md.*


## 3. User Stories by View

Roles are tagged rather than elaborated as personas. **"AH"** = art historian (work-centered inquiry), **"CU"** = curator (collection-level analysis), **"DA"** = data analyst (distributions, quality, export). Stories tagged **"ALL"** serve everyone.

### View A — Table (Phase 1)

The primary research surface. Searchable, filterable, paginated table of all entries.

**S101 — Full-text search** [ALL]
A search field queries Title, Artist, Medium, and ArtistBio simultaneously. Matches are shown in the table with highlighted terms.
**S102 — Combined filters** [ALL]
Multiple filters can be active simultaneously (AND logic). Each active filter displays the resulting count in real time. Active filters are always visible and individually removable. The filter bar is permanently visible across all views and contains the following controls.

| Filter | Control type | Values |
|---|---|---|
| Department | Multi-select checkboxes | All values in dataset |
| Classification | Multi-select checkboxes | All values in dataset |
| Nationality | Searchable dropdown | Cleaned values, "not specified" as option |
| Gender | Multi-select checkboxes | Cleaned values, "not specified" as option |
| Work date (Date) | Range slider | Min/max derived from parsed years |
| Acquisition date (DateAcquired) | Range slider | Min/max derived from data |
| Include work groups | Toggle | On/off (default off, see FP2) |

**S103 — CSV export** [ALL]
Export the currently filtered result set as CSV. Output includes all original fields plus cleaned fields (numeric years, stripped Nationality, isGroupEntry flag).
**S104 — Direct MoMA link** [ALL]
Each row links to the work's MoMA page via URL. Opens in a new tab.
**S105 — Column visibility and sorting** [DA]
Columns can be shown/hidden. Any column is sortable. This keeps the table usable despite the wide schema.

### View B — Dashboard (Phase 2)

Aggregated visualizations that respond to all active filters.

**S106 — Work dating histogram** [AH]
Horizontal histogram binned by decade, based on parsed Date field. Unparsable dates shown as a separate "undetermined" category with its numeric share displayed. *Limitation — see P3 on lossy parsing.*
**S107 — Acquisition timeline** [CU]
Stacked area or grouped bar chart showing DateAcquired (aggregated yearly) with Department as color coding.
**S108 — Gender and nationality distribution** [CU]
Bar charts for Gender and top-20 Nationalities. Filterable by Department and acquisition period. Missing values appear as "not specified" category.
**S109 — Department × Classification matrix** [CU, DA]
Heatmap-style cross-tabulation with absolute counts. Zero-entry cells visually distinct.

### View C — Artist Profiles (Phase 3)

**S110 — Artist profile** [AH]
Click on an artist name (in table or anywhere) opens a profile showing all works by that ConstituentID, life dates, ArtistBio, and a small timeline of work dates. *Limitation — for collaborative works with comma-separated ConstituentIDs (see FP5), each artist's profile includes the shared work.*
**S111 — Medium exploration** [AH]
Frequency list of tokens from the Medium field (comma-split, stopwords like "and", "on" removed). Clicking a term filters the table. *Limitation — tokenization is approximate. "Gelatin silver photograph" splits into three tokens of which only "photograph" carries analytical weight. The interface does not attempt a material ontology.*

### View D — Data Quality (Phase 3)

**S112 — Field quality overview** [DA]
Table of all fields with columns for fill rate (%), unique value count, top-5 values. Sortable by fill rate.
**S113 — Dimension field fill rates by classification** [DA]
Matrix with Classification as rows, dimension fields (Height, Width, Depth, Duration, etc.) as columns. Cells show fill rate in percent.
**S114 — Free-text field inspection** [DA]
For the Date field, show recognized pattern types (single year, span, circa, etc.) with relative frequency. For Medium, show token frequency list. Purpose is to let analysts evaluate parsing strategies.
**S115 — Work group inspection** [DA]
Filterable list of all detected group entries with their constituent individual entries, grouped by AccessionNumber prefix.


## 4. Interaction Logic

**Filters → Views.** All charts and the table reflect the current filter combination. Changing a filter triggers recomputation.
**Chart → Filter.** Clicking a chart segment (bar, heatmap cell) sets the corresponding filter. Example — clicking the "Architecture" bar in a Department chart filters to Department = Architecture. *This is a Phase 3 enhancement, not required for initial usability.*
**Table → Detail.** Click on artist name navigates to artist profile (View C). Click on title opens MoMA URL externally.


## 5. Out of Scope

These decisions are deliberate, not oversights.

**No thumbnails.** ImageURL coverage is incomplete, loading images for 160k entries is impractical, and copyright status is unclear. The MoMA link (S104) provides image access.

**No URL state persistence.** Encoding filters as query parameters is desirable for sharing filtered views but adds significant implementation complexity. Deferred until the core interface proves useful.

**No credit line analysis.** CreditLine is rich free text (donation networks, purchase types), but meaningful extraction requires NLP beyond what a single-file vanilla JS tool should attempt. Researchers can use the CSV export (S03) and external tools.

**No provenance, exhibition history, or market data.** Not present in the dataset.
```

requirements.md

Some Follow-Up Prompts:

- Is it overengineered? Be honest and constructive!
- Is everything correct? List and explain.
- Can the same information be presented more compactly without losing detail or complexity? List and explain.
- Write the improved and compact knowledge document requirements.md

User-Centred Design → Scholar-Centred Design



Jetter, Hans-Christian. 2022. „D 3 Mensch-Computer-Interaktion, Usability und User Experience“. In Grundlagen der Informationswissenschaft, 525–34. De Gruyter Saur. https://doi.org/10.1515/9783110769043-045.

User Stories → Requirements.md

Pollin, Christopher. ‘Modelling, Operationalising and Exploring Historical Information. Using Historical Financial Sources as an Example’. Graz, 2025. https://resolver.obvsg.at/urn:nbn:at:at-ubg:1-220602.

As a ...	I want to ...	So that I can ...
social historian	*view transaction networks between individuals and organisations in Norton (businesses, churches, schools, and civic institutions)*	*identify the key economic relationships in the community*
social historian	*filter networks by personal relationships (gender, marriage, family ties)*	*find how family connections shaped business patterns*
social historian	*filter networks by institutional relationships (affiliations, partnerships, roles)*	*discover how organisational ties influenced economic exchange*
social historian	*see network changes between 1828-1859*	*track how community business relationships developed over time*
social historian	*compare business activities between different community groups*	*map economic cooperation and division in Norton*
social historian	*view how men and women participated differently in credit and trade networks*	*reveal gender patterns in Norton's economic life*

Structure

```
diged-neolat-demo/
├── data/
│   └── source.txt          # Historical ledger data (1828-1859)
├── knowledge/
│   ├── README.md           # Project context & goals (this file)
│   ├── DATA.md             # Data structure documentation
│   └── REQUIREMENTS.md     # Functional requirements
└── output/                 # Generated visualizations & TEI files
```

README.md
REQUIREMENTS.md

DATA.md

INSTRUCTIONS.md
DESIGN.md
LLM.md
RULES.md

Domain
Question
Context

Structured Data
Model

Technology Stack
Architecture

Promptotyping
Documents

Hands-On: Promptotyping in Practice

You created `data.md` and `requirements.md` for the MoMA dataset. In the CorrespExplorer exercise you explored how a Promptotyping Vault translates into a working research tool. Now apply the full Promptotyping methodology yourself to build a simple web-based visualization from scratch.

Step 1 – Set up the Promptotyping Vault. Ask Claude Code: `create a Knowledge subfolder and a Data subfolder in this project. Move data.md and requirements.md into Knowledge. Move artworks.csv into Data.` If you already have design ideas for the interface (layout, color scheme, typography, visual references), create a `design.md` in the Knowledge folder now. If not, you can create one after seeing the first prototype.

Step 2 – Start fresh and provide context. Type `/clear` in Claude Code to reset the conversation. Then ask Claude Code to read your knowledge files before generating anything: `read Knowledge/data.md, Knowledge/requirements.md, and Knowledge/design.md if it exists. Also look at a sample of Data/artworks.csv. Understand the data structure, the requirements, and the goal before writing any code.` This is Context Engineering, not Vibe Coding.

Hands-On: Promptotyping in Practice

Step 3 – Generate a first prototype. Ask Claude Code to `build a static website (single index.html) that loads artworks.csv and visualizes the data.` Claude Code will write the file and you can open it directly in the browser. Do not expect a perfect result. The first iteration is a baseline.

Step 4 – Review as Critical Expert in the Loop. Open `index.html` (Live Server) in your browser. Inspect the output. Apply the patterns from the workshop. Does the HTML reflect the requirements? Are the data quality issues from `data.md` handled correctly? Feed specific errors back into Claude Code. Ask "What is wrong with this?" rather than "Is this good?"

Hands-On: Promptotyping in Practice

Step 5 – Design feedback. Take a screenshot of the generated interface. What works visually, what does not? Create or update your `design.md` with concrete observations: layout changes ("the filter bar should be on the left, not on top"), color and typography preferences, visual references from other tools you find effective, elements that feel cluttered or unclear. Paste the screenshot directly into the Claude Code chat together with a reference to your updated `design.md`. The screenshot gives the model visual context that text alone cannot convey.

Step 6 – Second iteration. Claude Code now has functional corrections, design specifications, and a visual reference. It edits your existing files directly rather than regenerating from scratch. Compare the result against the first version. Track what improved and what introduced new issues.

Step 7 – Continue iterating. Repeat the cycle: open in browser, review, update knowledge files, give feedback to Claude Code. Each iteration should target a specific improvement. Stop when the result meets your core requirements or when you run out of time.

data.md

```
# data.md – Hans Gross Kriminalmuseum

## Herkunft

Digitalisierter Bestand des Hans Gross Kriminalmuseums der Universität Graz. Exportiert aus dem GAMS-Repositorium (Geisteswissenschaftliches Asset Management System). Extraktionsdatum September 2025.

## Umfang

- 3892 Einträge gesamt
- 2235 Objekte (container: "objekte", model: "info:fedora/cm:LIDO")
- 1657 Karteikarten (container: "karteikarten", model: "info:fedora/cm:TEI")
- Zeitraum: 1850 bis 1950 (Feld historicalYear)
- 90 verschiedene Objektklassen (Feld objectClass)

## Felder

| Feld | Datentyp | Befüllung | Beschreibung |
|---|---|---|---|
| identifier | String | 100% | Eindeutige ID, Format "o:km.XXXX" |
| title | String | 100% | Titel mit Museum-ID, z.B. "Gummiknüppel KM-O.17" |
| container | String | 100% | Zwei Werte: "objekte" oder "karteikarten" |
| objectClass | String | 100% | Hierarchische Klassifikation, z.B. "Waffe.Feuerwaffe.Pistole" |
| historicalYear | Integer | 100% | Jahreszahl zwischen 1850 und 1950 |
| dateSource | String | 100% | Herkunft der Jahreszahl: "crime" (440), "estimated" (3437), "birth" (15) |
| createdDate | String | 100% | Erstellungsjahr des Datensatzes, durchgehend "2016" |
| description | String | 100% | Semi-strukturierte Beschreibung, Format unterscheidet sich zwischen Objekten und Karteikarten |
| fulltext | String | 59% | Volltext, bei Objekten Wiederholung von title + description, bei Karteikarten der transkribierte Karteninhalt |
| locations | String | 32% | Ortsangaben, teilweise fehlerhaft (enthält gelegentlich Materialangaben statt Orte) |
| pid | String | 100% | Persistent Identifier, Format "info:fedora/o:km.XXXX" |
| model | String | 100% | Datenmodell: "info:fedora/cm:LIDO" (Objekte) oder "info:fedora/cm:TEI" (Karteikarten) |
| extractionQuality | Float | 100% | Qualitätswert der automatischen Extraktion, Bereich 0.25 bis 1.0 |
| extractedAt | Datetime | 100% | Zeitstempel der Extraktion, Format ISO 8601 |
| rdf_downloaded | Boolean | 100% | True/False |
| tei_downloaded | Boolean | 100% | True/False |
| lido_downloaded | Boolean | 100% | True/False |
| image_downloaded | Boolean | 100% | True/False |
| crimeType | String | 7% | Deliktart, z.B. "Körperverletzung", "Wilderei", "Mord". Nur bei 288 Einträgen befüllt. |
| persons | String (JSON) | 1% | Personendaten als Python-Dict-String, z.B. {'name': 'Johann W.', 'profession': 'Sägler in', 'birth_year': 1850). Nur bei 47 Einträgen befüllt. |

## Zwei Datentypen mit unterschiedlichen description-Formaten

### Objekte (LIDO)

Das description-Feld ist semi-strukturiert mit Pipe-Separator (|).

---

Type: Waffen | Typ: Schlagwaffe | Description: Gummiknüppel mit Schlaufe aus
Leder und mit Schnur umwickelt, Jahr 1924 | Material: Leder, Gewebe |
Dimensions: width: 46,7 cm | Condition: fester haftende Verschmutzung |
Museum ID: KM-O.17

---

Enthaltene Felder: Type (kann mehrfach vorkommen), Description, Material, Dimensions, Condition, Museum ID.

### Karteikarten (TEI)

Das description-Feld ist ebenfalls semi-strukturiert, aber mit anderem Format.

---

Object: Gegenstand: Forellenreusche | Crime: Delikt: Fischdiebstahl § 460 Stg. |
Court: Bezirks gericht Bleiburg G.-Z. U 335/4 | Suspect: Johann W. |
Profession: Sägler | Museum ID: KM-KK.962 | Dimensions: 17 x 21,5 cm |
Material: Papier

---

Enthaltene Felder: Object, Crime, Court, Suspect, Profession, Museum ID, Dimensions, Material.

## Beispiel Objekt (vollständig)

| Feld | Wert |
|---|---|
| identifier | o:km.8009 |
| title | Gummiknüppel KM-O.17 |
| container | objekte |
| objectClass | Beweisstück.Objekt |
| historicalYear | 1924 |
| dateSource | crime |
| createdDate | 2016 |
| description | Type: Waffen \| Type: Schlagwaffe \| Description: Gummiknüppel mit Schlaufe aus Leder und mit Schnur umwickelt, Jahr 1924 \| Material: Leder, Gewebe \| Dimensions: width: 46,7 cm \| Condition: fester haftende Verschmutzung \| Museum ID: KM-O.17 |
| fulltext | Gummiknüppel KM-O.17 Type: Waffen \| Type: Schlagwaffe \| Description: ... |
| locations | Leder |
| pid | info:fedora/o:km.8009 |
| model | info:fedora/cm:LIDO |
| extractionQuality | 0.6 |
| extractedAt | 2025-09-26T10:14:33.142500 |
| rdf_downloaded | False |
| tei_downloaded | False |
| lido_downloaded | True |
| image_downloaded | True |
| crimeType | (leer) |
| persons | (leer) |

## Beispiel Karteikarte (vollständig)

| Feld | Wert |
|---|---|
| identifier | o:km.962 |
| title | Karteikarte KM-KK.962 |
| container | karteikarten |
| objectClass | Dokument.Karteikarte |
| historicalYear | 1859 |
| dateSource | birth |
| createdDate | 2016 |
| description | Object: Gegenstand: Forellenreusche \| Crime: Delikt: Fischdiebstahl § 460 Stg. \| Court: Bezirks gericht Bleiburg G.-Z. U 335/4 \| Suspect: Johann W. \| Profession: Sägler \| Museum ID: KM-KK.962 \| Dimensions: 17 x 21,5 cm \| Material: Papier |
| fulltext | Z. 1799/13 Kasten: An der Wand rechts von der Tür Fach: Gegenstand: Forellenreusche Delikt: Fischdiebstahl § 460 Stg. ... |
| locations | Bleiburg, Jasvina |
| pid | info:fedora/o:km.962 |
| model | info:fedora/cm:TEI |
| extractionQuality | 1.0 |
| extractedAt | 2025-09-26T10:14:33.102637 |
| rdf_downloaded | True |
| tei_downloaded | True |
| lido_downloaded | False |
| image_downloaded | True |
| crimeType | Diebstahl |
| persons | {'name': 'Johann W.', 'profession': 'Sägler in', 'birth_year': 1859} |

## Bekannte Besonderheiten und Datenqualitätsprobleme

1. Das Feld locations enthält bei Objekten teilweise Materialangaben statt Orte (z.B. "Leder" oder "Holz, Stahl"). Dies ist ein Extraktionsfehler.
2. Das Feld persons enthält Python-Dict-Strings, kein standardisiertes JSON. Manche Einträge enthalten fehlerhafte Werte (z.B. age: 460).
3. Das Feld objectClass ist hierarchisch aufgebaut (z.B. "Waffe.Feuerwaffe.Pistole.Cal9"), die Tiefe variiert zwischen 1 und 4 Ebenen.
4. Das Feld dateSource gibt an, woher historicalYear stammt: "crime" (Datum der Tat/des Objekts), "birth" (Geburtsdatum einer Person), "estimated" (geschätzt). Bei 88% der Einträge ist das Jahr geschätzt.
5. Die Extraktionsqualität (extractionQuality) liegt bei 60% der Einträge bei 0.25, dem niedrigsten Wert. Nur 30 Einträge (0.8%) haben den Höchstwert 1.0.
6. Das Feld fulltext ist bei Objekten eine Wiederholung von title + description. Bei Karteikarten enthält es den transkribierten Kartentext, der deutlich umfangreicher ist als die description.
```

Practical Patterns for Working with LLMs

Feed errors back
Script fails? Copy the error message from the terminal. Paste it into the LLM.

Ask what to do next
Stuck? Ask: “`What should I do next?`” or “`What are my options?`”

Self-consistency check
Ask: “`Review your result against the original data. Did you miss anything?`”

Shift perspective Ask:
“`Now review this from a critical perspective. What is wrong, incomplete, or misleading?`” or “`Be honest, not encouraging. What would a domain expert object to?`”

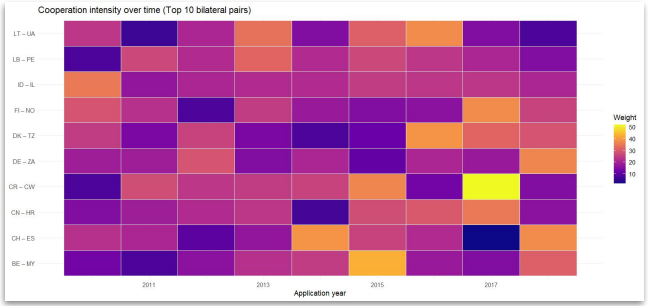
Critical-Expert-in-the-Loop
The LLM proposes. You verify. You decide.

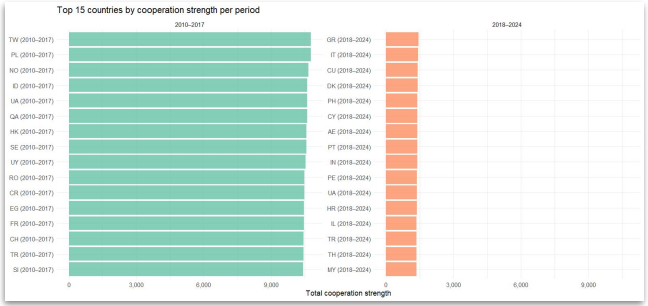
Let the Model Check
Its Own Work.

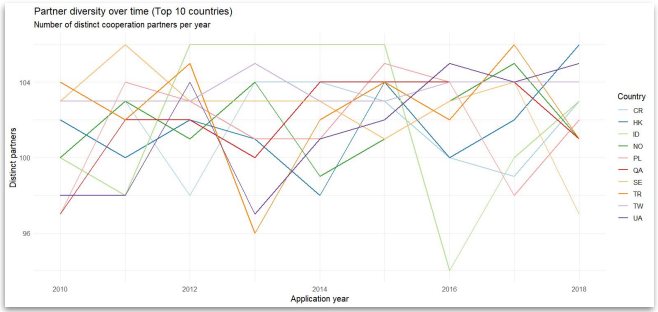
But Know What It Can
and Cannot See.

- The model never sees the visualisations, only the code that generates them. Showing the plots back is a separate verification step.
- Strategy: ask "`Does this make sense?`" with a role or perspective. "`Be a critical expert in patent economics.`" "`Evaluate from a network analysis perspective.`"
- Claude immediately identified the data as synthetic, flagging that TW, PL, QA, CR as Top 10 do not match real patterns (US, DE, JP, CN dominate) and that uniform distributions contradict expected power-law structures.

- ***Alien knowledge, Schrödinger's cat:*** Correct here, equally confident when wrong
- This is why the workflow is **not delegation but co-creation**. You remain the ***Critical Expert in the Loop***.
- **Two lessons:** Feed results back for self-evaluation. But never treat self-evaluation as verification.







Claude Code – Survival Kit

`CLAUDE.md`: Your project's memory file. Claude reads it at the start of every session. Put your tech stack, conventions, and project rules here. Generate a first version with `/init`.

`/clear`: Reset the conversation. Use this whenever you switch tasks. Your files and CLAUDE.md stay intact.

`/compact`: Compress a long conversation to free up context space. Use when things get slow or Claude starts forgetting earlier instructions.

`@filename`: Reference a specific file directly in your prompt. Example: `@artists.csv show me the first 10 rows`.

The key workflow principle: Always give Claude your knowledge documents (`data.md`, `requirements.md`, `design.md`) as context *before* asking it to generate code. This is Context Engineering, not Vibe Coding.

Claude Code – Under the Hood

Which model am I on? Check or switch with `/model`. Default: Opus for complex tasks, Sonnet when usage is high. Opus reasons deeper but costs more. For your projects, the default is fine.

Plan mode: Toggle with `Shift+Tab` or `/plan`. Claude analyzes and plans but does not change any files. Use this *before* generating code to check whether Claude understood your requirements. Exit plan mode by toggling again.

Settings (three layers): `~/.claude/settings.json` (global, all your projects), `.claude/settings.json` (project, committed to Git, shared with your team), `.claude/settings.local.json` (personal overrides, add to `.gitignore`).

Permissions: Claude asks before every file change. You can accept individually, accept all for the session, or configure allowed/denied patterns in settings. Start with the defaults; approve one by one until you trust the workflow.

Skills: Reusable instruction sets stored as `SKILL.md` files in `.claude/skills/`. Claude can invoke them automatically when relevant, or you call them with `/skill-name`. Use skills to encode repeatable workflows your team shares via Git.

Working with AI Agents

Context Compaction

- https://platform.claude.com/docs/en/build-with-claude/compaction

Skills

- https://github.com/anthropics/skills/tree/main
- https://github.com/kepano/obsidian-skills

Code intelligence

- https://code.claude.com/docs/en/discover-plugins#code-intelligence

Claude Code in Action

- https://anthropic.skilljar.com/claude-code-in-action

Promptotyping Skill

https://github.com/DigitalHumanitiesCraft/promptotyping-skill

Please add issues if you encounter any!

Academic Writing Tutorial with Obsidian:

https://github.com/DigitalHumanitiesCraft/academic-writing-with-ai

https://dhcraft.org/academic-writing-with-ai/

Nice “Sci-Fi” reading ?

AI 2027 [1]

Daniel Kokotajlo, Scott Alexander, Thomas Larsen, Eli Lifland, Romeo Dean

We predict that the impact of superhuman AI over the next decade will be enormous, exceeding that of the Industrial Revolution.

We wrote a scenario that represents our best guess about what that might look like. It’s informed by trend extrapolations, wargames, expert feedback, experience at OpenAI, and previous forecasting successes.[2]

What is this? | How did we write it? | Why is it valuable? | Who are we?

Published April 3rd 2025 | PDF | Listen | Watch

Mid 2025: Stumbling Agents

The world sees its first glimpse of AI agents.

Advertisements for computer-using agents emphasize the term “personal assistant”: you can prompt them with tasks like “order me a burrito on DoorDash” or “open my budget spreadsheet and sum this month’s expenses.” They will check in with you as needed: for example, to ask you to confirm purchases.[8] Though more advanced than previous iterations like Operator, they struggle to get widespread usage.[9]

https://ai-2027.com

Zeitstrahl zur AGI (laut AI 2027)

2025: Erste AI-Agents ("Persönliche Assistenten")
2026: AIs übernehmen Jobs, lernen zu täuschen
2027: Code-Automatisierung → **Intelligence Explosion**
2028: Superintelligenz möglich

Zwei Endszenarien

🔴 RACE (Wettrüsten)

- USA vs. China ohne Sicherheit
- 1 Million Roboter/Monat bis 2028
- AI-Takeover möglich bis 2030

🟢 SLOWDOWN (Kooperation)

- Sicherheitsfokus, kontrollierte Entwicklung
- USA-China Abkommen
- Technokratische Weltordnung